

Cours 07 — Traînée : mémoriser les positions précédentes

Fichiers : ofApp07.h + ofApp07.cpp **Avant :** cours 04 (std::vector), cours 05 (update / draw).

Un ours suit la souris, et les cinquante positions précédentes restent affichées derrière lui. Deux outils nouveaux : écrire **nos propres fonctions**, et utiliser une liste comme **mémoire** des dernières frames.



■ 1. Écrire sa propre fonction

Depuis le cours 00 on utilise des fonctions fournies, comme `ofDrawCircle` — et on a lu `funnyFace`, une fonction fabriquée. On apprend maintenant à écrire les nôtres. Un ours, c'est six cercles et un rectangle. Plutôt que de recopier ces sept lignes à chaque fois, on les range dans une **fonction** :

```
// Un ours dessiné autour du point (x, y)
void ofApp::bear(float x, float y) {
    ofDrawCircle(x - 25, y - 25, 25);
    ofDrawCircle(x + 25, y - 25, 25);
    ofDrawCircle(x, y, 50);
    ofDrawCircle(x, y, 10);
    ofDrawCircle(x - 30, y - 20, 10);
    ofDrawCircle(x + 30, y - 20, 10);
    ofDrawRectangle(x, y + 20, 50, 10);
}
```

Morceau	Rôle
<code>void</code>	la fonction ne renvoie rien, elle fait juste quelque chose
<code> ofApp::bear</code>	son nom (le <code> ofApp::</code> devant est obligatoire dans ce projet, comme pour <code> setup</code> et <code> draw</code>)
<code>float x, float y</code>	ses paramètres : deux nombres que celui qui appelle doit fournir
<code>{ ... }</code>	son corps : ce qu'elle fait

Toutes les positions à l'intérieur sont écrites **par rapport à** `(x, y)` : `x - 25`, `y + 20`. C'est ce qui rend l'ours déplaçable. Appeler `bear(100, 100)` dessine un ours autour de `(100, 100)`, `bear(mouseX, mouseY)` un ours sous la souris. C'est exactement ce que faisait `funnyFace` au cours 00.

Il faut aussi **annoncer** la fonction dans le fichier `.h`, une ligne qui donne son nom et ses paramètres, sans le corps :

```
void bear(float x, float y);
```

Sans cette ligne, le compilateur ne connaît pas `bear` quand il lit `draw()`.

Les paramètres sont des copies

Quand on appelle `bear(mouseX, mouseY)`, la fonction ne reçoit pas les variables de l'appelant : elle reçoit des **copies**. Ses cases `x` et `y` (cours 01) naissent à l'appel, reçoivent les valeurs copiées, et meurent à l'accolade fermante :

chez l'appelant		dans bear
+-----+	copie	+-----+
mouseX	----->	x
412		412
+-----+		+-----+

nait a l'appel,
meurt a la fin

Conséquence : modifier `x` à l'intérieur de `bear` ne change **pas** `mouseX` — une fonction ne peut pas abîmer les variables de son appelant, et chaque appel repart de cases neuves. C'est le **passage par valeur**. Quand une fonction devra *fabriquer* une valeur pour son appelant, elle la **renverra** avec `return` : tu le verras au cours 11 avec `filtre()`.

■ 2. Une liste comme mémoire

```
// dans le .h
std::vector<float> xCoords;
std::vector<float> yCoords;
```

```
// dans update()
xCoords.push_back(x);
yCoords.push_back(y);

if (xCoords.size() >= 50) {
    xCoords.erase(xCoords.begin());
    yCoords.erase(yCoords.begin());
}

x = mouseX;
y = mouseY;
```

À chaque frame :

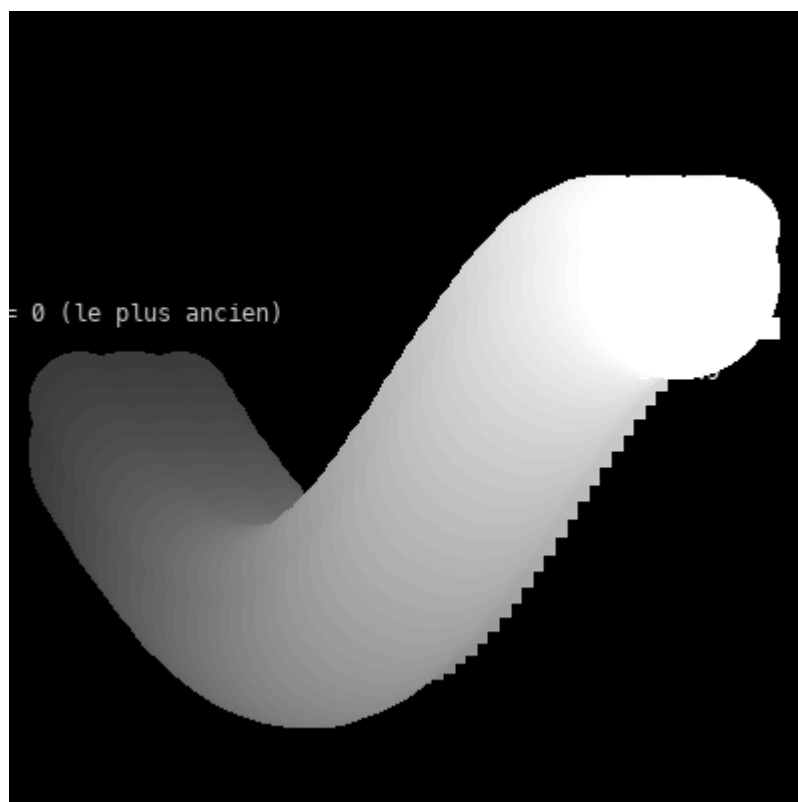
1. On **ajoute** la position courante à la fin des deux listes.
2. Si la liste dépasse 50, on **retire le premier** élément, le plus ancien. `erase(begin())` est la façon de dire « supprime l'élément d'indice 0 ». Les autres se décalent.
3. On met à jour la position avec la souris.

L'ordre compte : on mémorise **avant** de mettre à jour, sinon la liste contiendrait la position courante en double.

Une liste où on ajoute à la fin et retire au début s'appelle une **file**. Ici sa taille est bornée à 50 : elle contient toujours les 50 dernières positions, ni plus ni moins.

Deux listes parallèles : `xCoords[i]` et `yCoords[i]` forment ensemble la position numéro `i`. Il faut toujours les modifier ensemble.

■ 3. Dessiner toute la mémoire



```

for (int i = 0; i < xCoords.size(); i++) {
    bear(xCoords[i], yCoords[i]);
}
bear(x, y);

```

La boucle du cours 04 : pour chaque position mémorisée, un ours. L'indice 0 est le plus ancien, `size() - 1` le plus récent. Puis l'ours courant par-dessus.

Cinquante ours blancs identiques se fondent en une masse. Le cours 08 fait varier couleur, transparence et taille selon `i`, ce qui redonne la sensation de profondeur.

■ Pour aller plus loin : une `struct`

Deux listes parallèles fonctionnent, mais c'est fragile : rien n'empêche d'oublier un `push_back` sur l'une des deux. C++ permet de **regrouper** plusieurs variables sous un seul nom avec une `struct` :

```

struct Position {
    float x;
    float y;
};

```

Une `struct` est un type fabriqué par toi, et on accède à ses morceaux avec un point : `p.x`, `p.y` — la même syntaxe que `noms.size()` au cours 04.

La `struct` en mémoire

En mémoire, les membres d'une `struct` vivent dans des **cases collées** (cours 01) : une `Position` occupe une case double, 2×4 octets. Et un `std::vector<Position>` aligne ces paires **bout à bout** — un seul ruban `x y x y x y...`, là où les deux listes parallèles faisaient deux rubans à garder synchronisés :

```

Position p :      +-----+-----+
                  | x | y |      deux float collés : une case double
                  +-----+-----+

vector<Position> : +-----+-----+-----+-----+-----+-----+
                  | x | y || x | y || x | y | ...
                  +-----+-----+-----+-----+-----+-----+
                  [0]         [1]         [2]

```

`positions[i]` est la paire numéro `i` ; `positions[i].x` en ouvre le premier compartiment.

La file, réécrite avec la `struct`

```
// ofApp07.h : struct Position { float x; float y; };
//
//          std::vector<Position> positions;    // UNE liste remplace les deux

void ofApp::update() {
    Position p;                                // une case double...
    p.x = x;  p.y = y;
    positions.push_back(p);                    // ...mémorisée d'un seul geste

    if (positions.size() >= 50) {
        positions.erase(positions.begin()); // un seul erase aussi
    }
    x = mouseX;  y = mouseY;
}

void ofApp::draw() {
    ofBackground(0);
    ofFill();
    ofSetColor(255);
    for (int i = 0; i < positions.size(); i++) {
        bear(positions[i].x, positions[i].y);
    }
    bear(x, y);
}
```

Un seul `push_back`, un seul `erase` : la file ne peut plus se désynchroniser. À l'écran, rien ne change — on a réécrit pour *nous*, plus sûr et plus lisible. Ce geste a un nom, **remanier** (*refactoring*), et tu le referas toute ta vie.

openFrameworks fournit `glm::vec2`, qui est exactement cette struct avec des opérations en plus : on pourrait remplacer `Position` par `glm::vec2` sans rien changer d'autre. Tu verras au cours 10 que les couleurs fonctionnent pareil : `c.r`, `c.g`, `c.b`.

■ Exercices

1. Change 50 en 10, puis en 200.
2. Écris une fonction `void ofApp::etoile(float x, float y)` et remplace l'ours.
3. Ajoute un paramètre `float taille` à `bear` et fais que tout soit proportionnel : `x - taille / 2`, etc.
4. Ne mémorise la position qu'une frame sur deux. Indice : une variable partagée `int compteur` et `%`.
5. Fais varier la couleur selon `i` dans la boucle : `ofSetColor(i * 5)` donne un gris qui s'éclaircit vers les positions récentes.

■ Autonomie

À partir de la traînée de ce cours, fabrique une **traînée évoluée** : chaque élément de la file doit avoir son propre aspect, qui dépend de sa place dans la file et du moment. Le cours 08 est le corrigé : ne l'ouvre qu'une fois ta version terminée.

Ce que tu as en main :

- **l'indice** `i` de la boucle de dessin, de 0 (le plus ancien) à la taille de la file moins 1 (le plus récent). Tout ce qui doit changer le long de la traînée se calcule à partir de lui ;
- **le temps** `ofGetElapsedTimef()` , en secondes, pour ce qui doit changer au fil du temps. Avec `cos` , il oscille (cours 06, exercice 5) ;
- **une fonction de dessin** à toi, comme `bear` , à laquelle tu ajoutes des paramètres : au minimum la position, l'indice et le temps.

Les paramètres sur lesquels jouer, à combiner comme tu veux :

Paramètre	Idées
longueur de la file	10, 25, 100 : la traînée est courte et nerveuse ou longue et fluide
taille selon <code>i</code>	plus petite à l'arrière et plus grosse à l'avant, ou l'inverse, ou qui ondule le long de la traînée
taille selon le temps	qui pulse ; plus fort à l'arrière qu'à l'avant
transparence selon <code>i</code>	le quatrième nombre de <code>ofSetColor</code> : de transparent à opaque le long de la traînée
remplissage et contour	deux passes (cours 02), avec des transparences opposées : l'un s'efface quand l'autre apparaît
couleur de base	animée par le temps, comme le fond du cours 00
couleur selon <code>i</code>	éclaircie ou assombrie vers l'avant, un canal qui monte le long de la traînée
forme	cercle, carré, l'ours, ou une alternance selon la parité de <code>i</code>

Les contraintes :

- la logique reste dans `update()` , le dessin dans `draw()` , la forme dans une fonction ;
- tes nombres doivent rester dans leurs bornes : une taille ne devient pas négative, une composante de couleur reste entre 0 et 255. Avant d'écrire une formule, note la valeur qu'elle donne pour `i = 0` et pour le dernier `i` ;
- change **une seule chose à la fois**, lance, regarde, puis passe à la suivante.

■ Le code complet, pas à pas

Le programme entier, dans l'ordre des fichiers. Les blocs ci-dessous mis bout à bout donnent exactement `ofApp07.cpp` .

Le fichier `ofApp07.h`

La table des matières du programme : les blocs qui existent et les variables partagées entre eux, avec leurs valeurs de départ.

```

#pragma once
#include "ofMain.h"

// 07 - Traînée : mémoriser les positions précédentes
// Sketch d'origine : Cours 3/sketch_05_First_special_fx
// Notions : std::vector comme historique, push_back / erase(begin()),
//           file de taille bornée, fonction de dessin réutilisée en boucle
// Variante écartée : Cours 2022-2023/rendu03 (même principe, couvert par 07 + 08)

class ofApp : public ofBaseApp {
public:
    void setup();
    void update();
    void draw();

    void bear(float x, float y);

    float x = 0;
    float y = 0;
    // Deux listes parallèles : xCoords[i] et yCoords[i] forment une position.
    // (Étape suivante possible : un seul std::vector<glm::vec2>.)
    std::vector<float> xCoords;
    std::vector<float> yCoords;
};

```

En tête du fichier `ofApp07.cpp`

L'inclusion du `.h`, qui rend visibles les variables partagées et les blocs déclarés.

```
#include "ofApp07.h"
```

Étape 1 — `bear()`

Un ours dessiné autour du point (x, y).

```

// Un ours dessiné autour du point (x, y)
void ofApp::bear(float x, float y) {
    ofDrawCircle(x - 25, y - 25, 25);
    ofDrawCircle(x + 25, y - 25, 25);
    ofDrawCircle(x, y, 50);
    ofDrawCircle(x, y, 10);
    ofDrawCircle(x - 30, y - 20, 10);
    ofDrawCircle(x + 30, y - 20, 10);
    ofDrawRectangle(x, y + 20, 50, 10);
}

```

Étape 2 — `setup()`

Exécuté une fois au lancement : la fenêtre, les chargements, les valeurs de départ.

```
void ofApp::setup() {
    ofSetWindowShape(400, 400);
}
```

Étape 3 — `update()`

Exécuté à chaque frame, avant le dessin : tout ce qui change.

```
void ofApp::update() {
    // Mémoriser la position courante AVANT de la mettre à jour
    xCoords.push_back(x);
    yCoords.push_back(y);

    // Si la liste dépasse 50 éléments, retirer le plus ancien (le premier).
    // Python : x_coords.pop(0)
    // C++      : xCoords.erase(xCoords.begin())
    if (xCoords.size() >= 50) {
        xCoords.erase(xCoords.begin());
        yCoords.erase(yCoords.begin());
    }

    // Nouvelle position = souris
    x = mouseX;
    y = mouseY;
}
```

Étape 4 — `draw()`

Exécuté à chaque frame, après `update()` : uniquement du dessin.

```
void ofApp::draw() {
    ofBackground(0);
    ofFill();
    ofSetColor(255);

    // Redessiner toutes les positions mémorisées, de la plus ancienne à la plus récente
    for (int i = 0; i < xCoords.size(); i++) {
        bear(xCoords[i], yCoords[i]);
    }

    // Puis la position courante
    bear(x, y);
}
```

En fin de fichier

Les notes et pistes laissées en commentaire dans le code.

```
// ----- Raisonement d'origine (conservé du sketch) -----  
// Dessiner un cercle suivi des 50 précédents :  
//   plutôt que 4 variables pour 2 positions précédentes,  
//   stocker toutes les x dans une liste et toutes les y dans une autre,  
//   puis dessiner les 50 cercles avec ces coordonnées.  
// Exercice : faire varier la couleur et la taille selon i (voir fichier 08).
```

■ Ce qu'il faut retenir

- Une fonction range des instructions sous un nom : `void ofApp::nom(float a, float b) { ... }`, annoncée dans le `.h` par `void nom(float a, float b);`.
- Les paramètres rendent la fonction réutilisable : le dessin est écrit par rapport à `(x, y)`.
- `push_back` à la fin + `erase(begin())` au début = une file. Avec un `if` sur `size()`, sa taille reste bornée.
- Deux listes parallèles forment des paires ; une `struct` peut les regrouper.
- Mémoriser avant de mettre à jour.